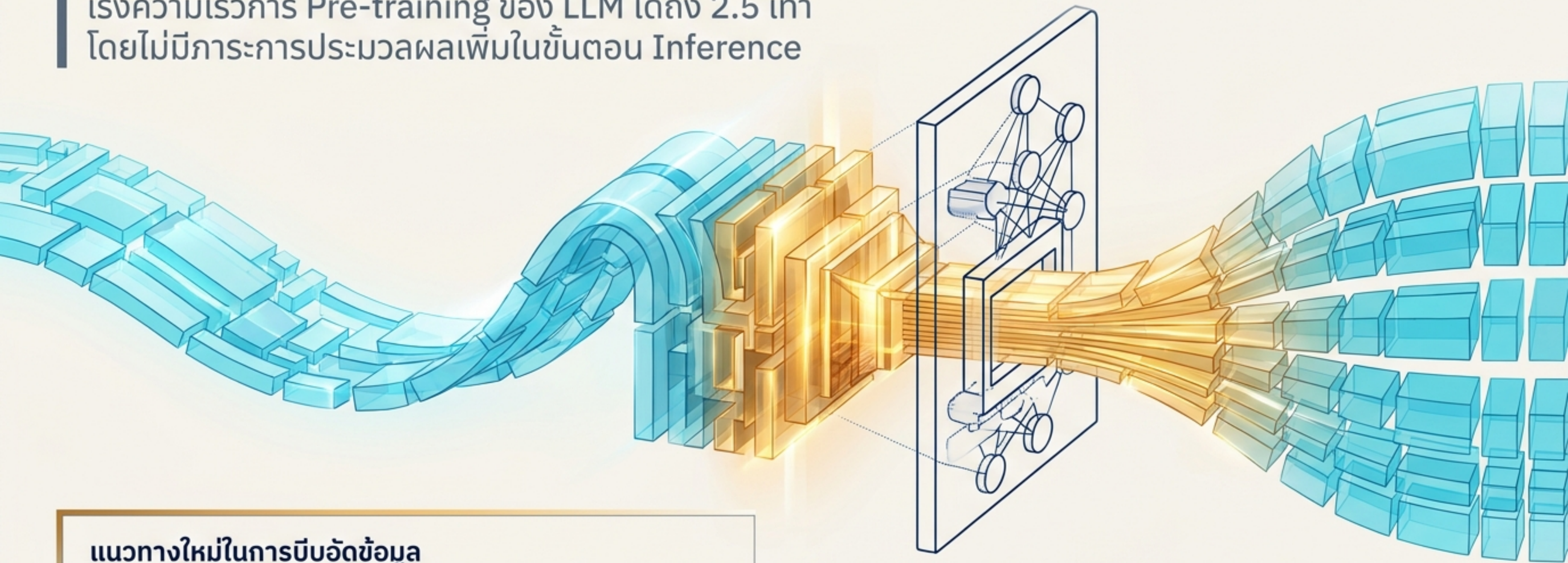


Token-Superposition Training (TST)

เร่งความเร็วการ Pre-training ของ LLM ได้ถึง 2.5 เท่า
โดยไม่มีภาระการประมวลผลเพิ่มในขั้นตอน Inference



แนวทางใหม่ในการบีบอัดข้อมูล (A New Paradigm for Data Compression)

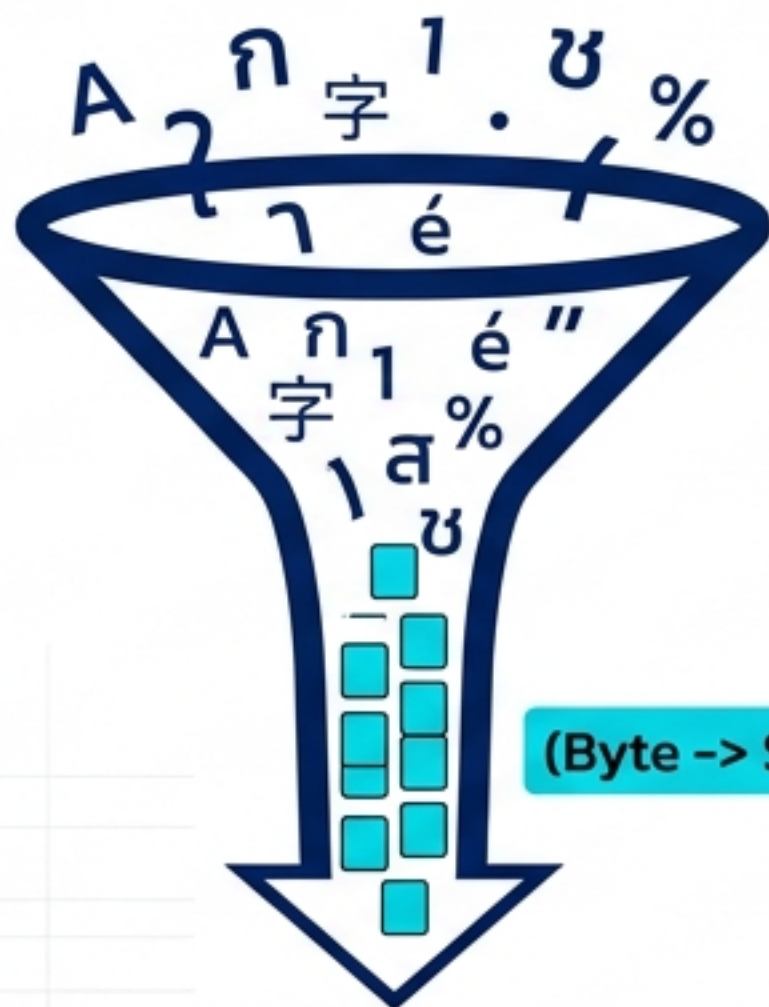
- นำเสนอเทคนิค Drop-in ที่ปรับเปลี่ยนเฉพาะชุดข้อมูลและ Loss Function โดยรักษาโครงสร้างสถาปัตยกรรมโมเดลไว้ดั้งเดิม 100%

ความท้าทาย: คอขวดของทรัพยากร Compute ในยุค Data-Hungry LLMs

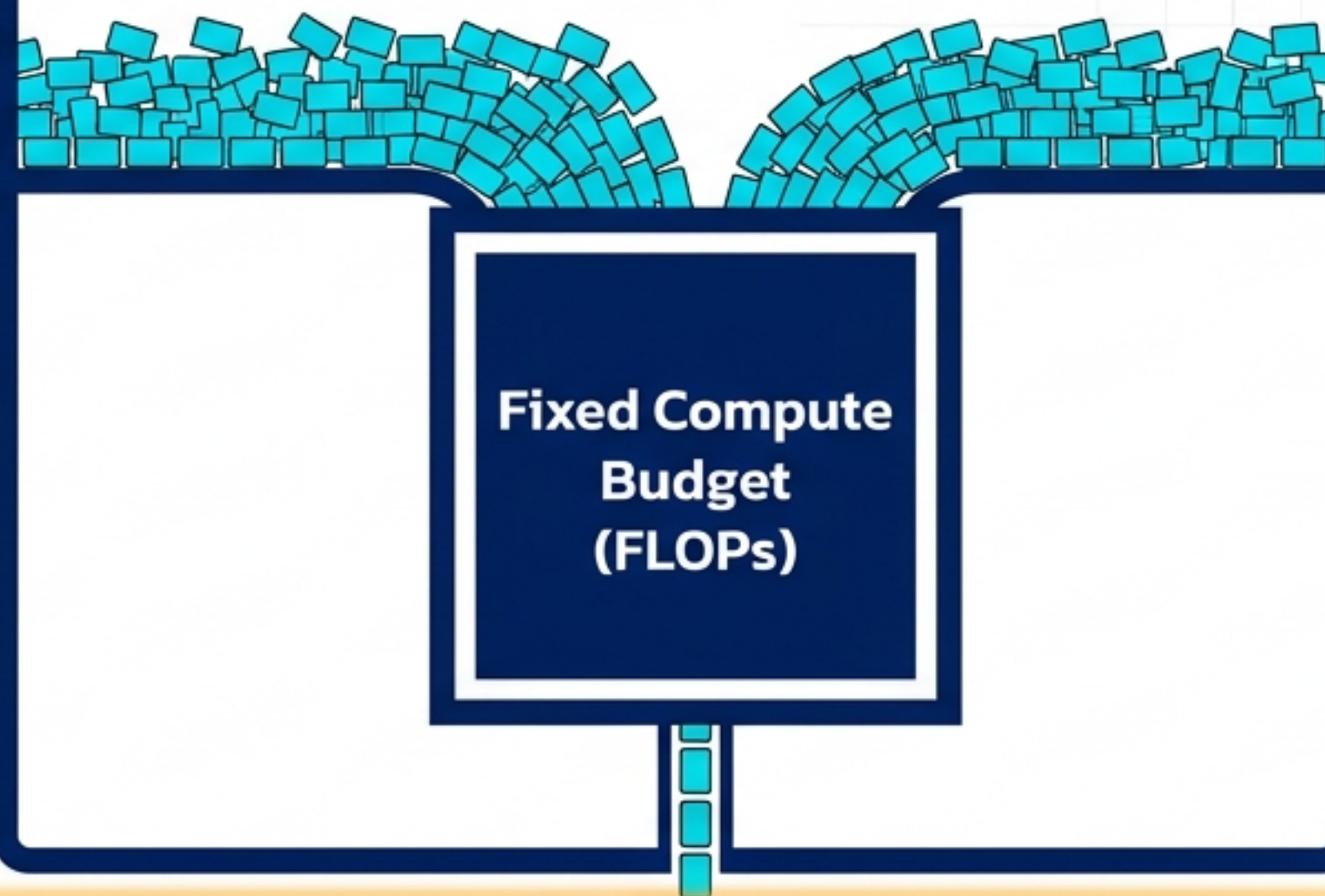
โมเดลภาษาในปัจจุบันถูกฝึกสอนเกินจุด Compute-optimal ไปมาก

ปัญหาหลักไม่ใช่จำนวนพารามิเตอร์ แต่คือ อัตราการนำเข้าข้อมูล ภายใต้ขีดจำกัดของ FLOPs

อดีต: เพิ่มประสิทธิภาพผ่าน Tokenizer

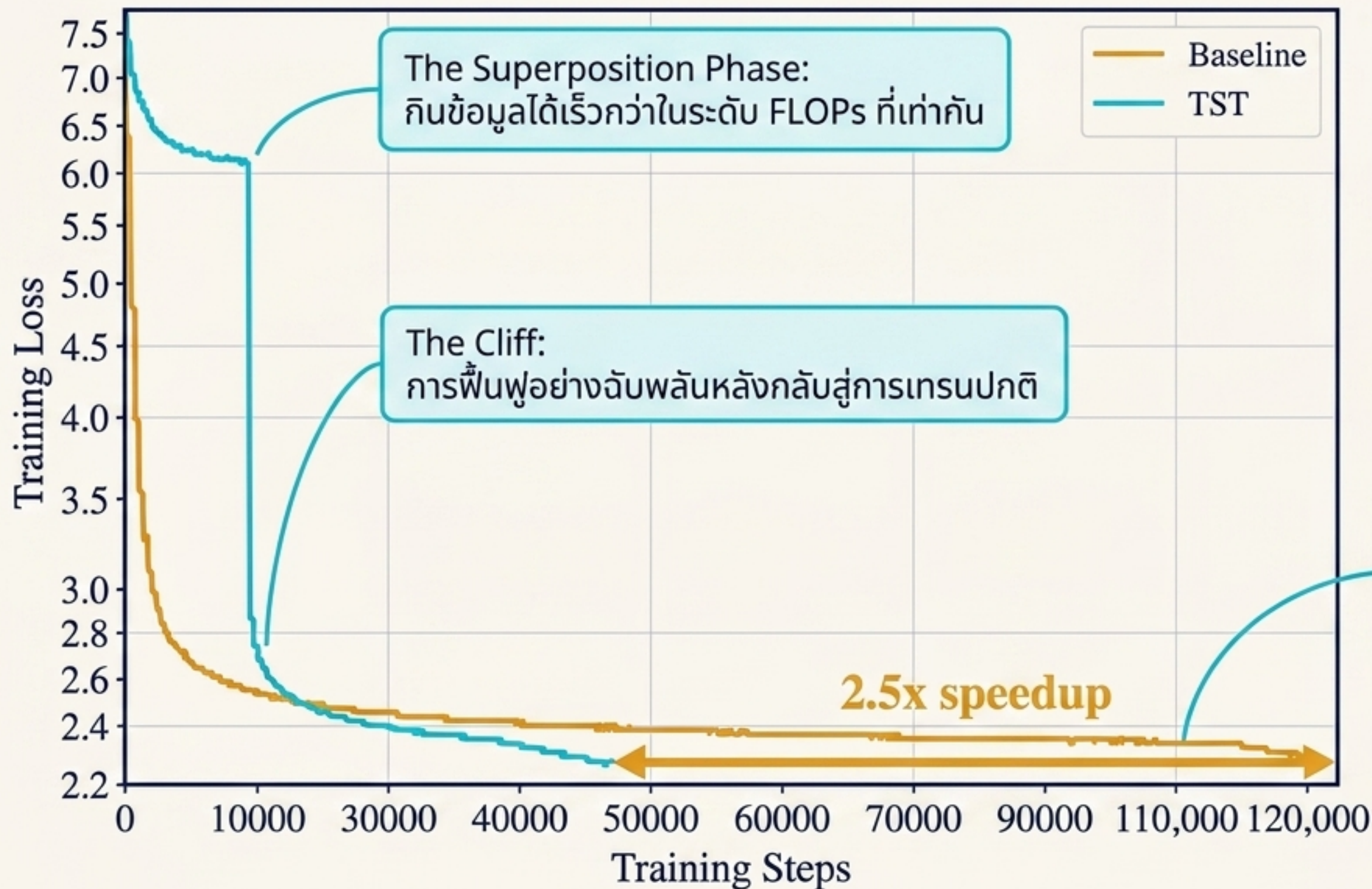


ปัจจุบัน: ขีดจำกัดของ Compute Budget



เป้าหมาย: ทำอย่างไรให้โมเดลประมวลผล Token ได้มากขึ้นต่อ 1 FLOP โดยไม่กระทบความสามารถในการทำงานจริง?

ผลลัพธ์: การเร่งความเร็ว 2.5x ในระดับโครงสร้าง 10B MoE

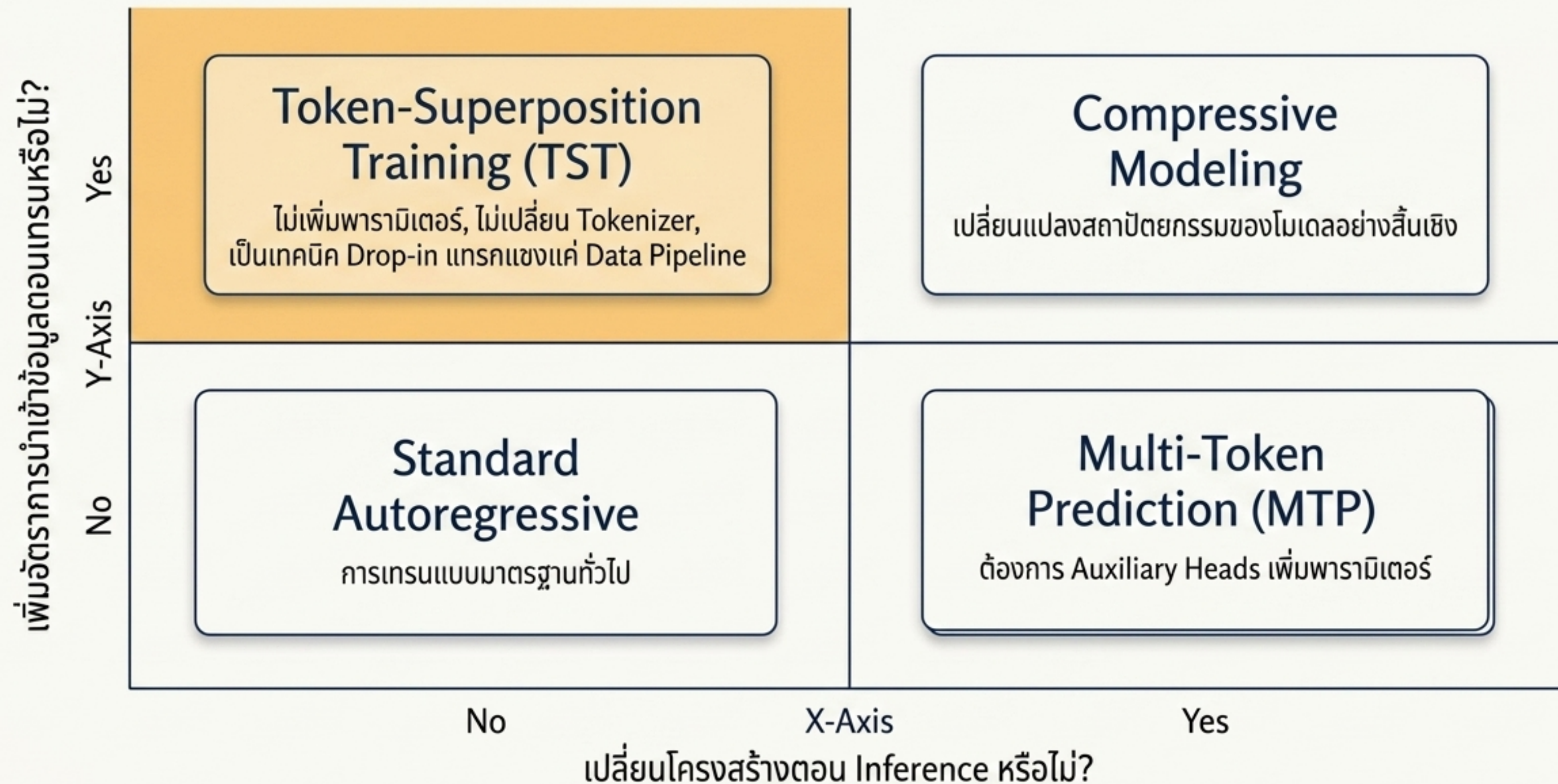


โมเดล: Qwen3-like MoE (10B-A1B)

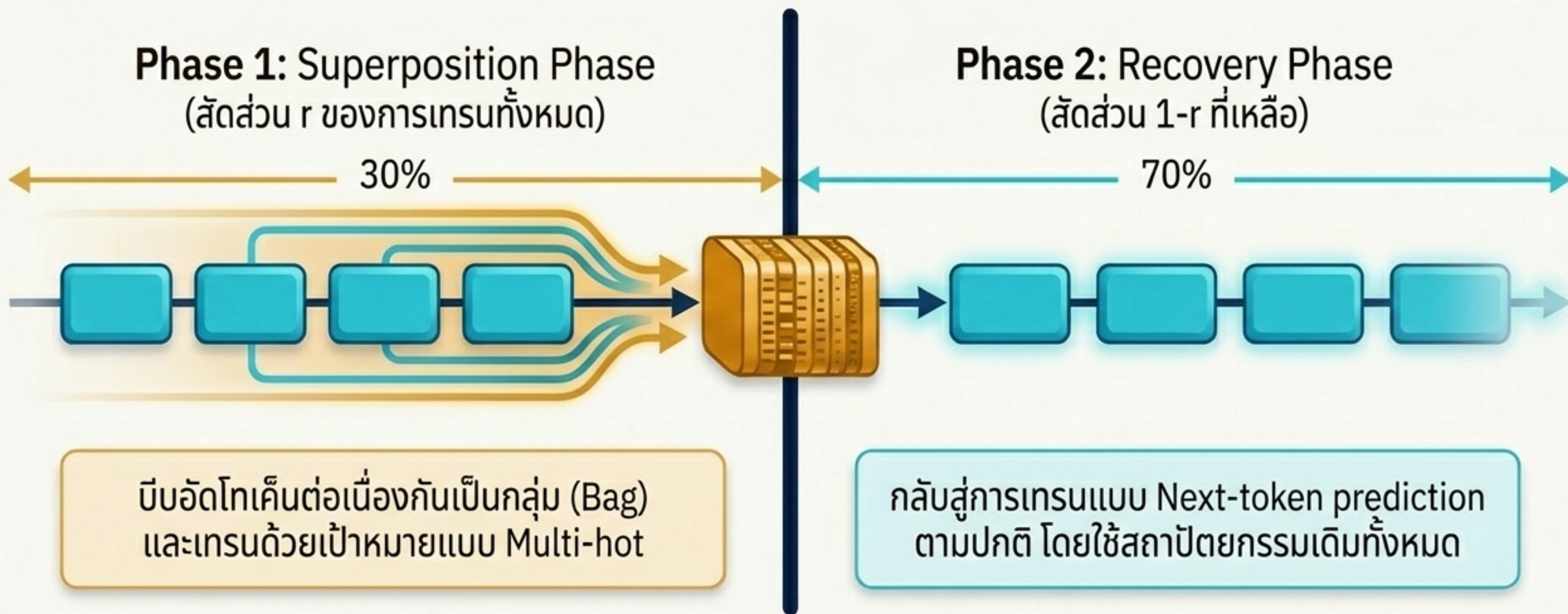
ผลลัพธ์: เข้าถึงค่า Training Loss ที่เทียบเท่า Baseline ได้เร็วกว่า 2.5 เท่า

ประหยัดเวลาและทรัพยากรการคำนวณอย่างมหาศาล (2.5x Speedup)

จุดยืนของ TST ในสมรภูมิความคุ้มค่าของการ Pre-training



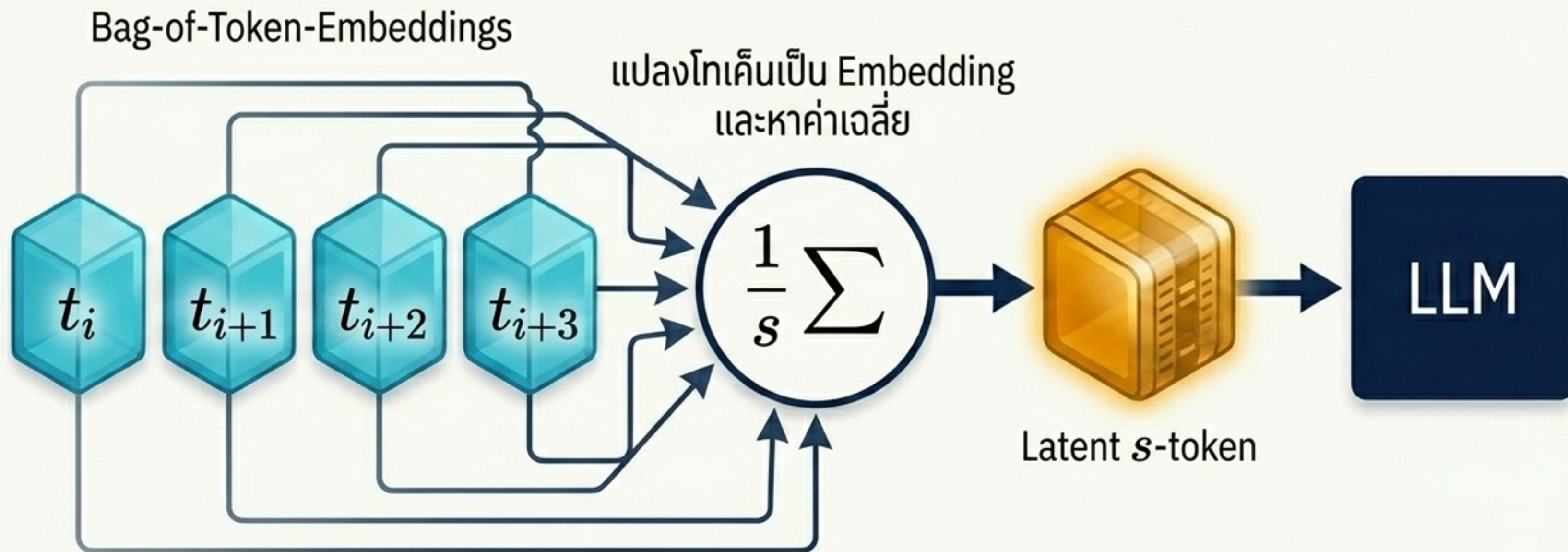
กลไกการทำงาน 2 ระยะ (The Two-Phase Architecture)



พารามิเตอร์ที่สำคัญ: ค่า s (Bag size) = จำนวนโทเค็นที่ถูกซ้อนทับ | ค่า r (Ratio) = สัดส่วนของ Step ที่ใช้ในระยาะซ้อนทับ

การซ้อนทับข้อมูลขาเข้า (Input Superposition)

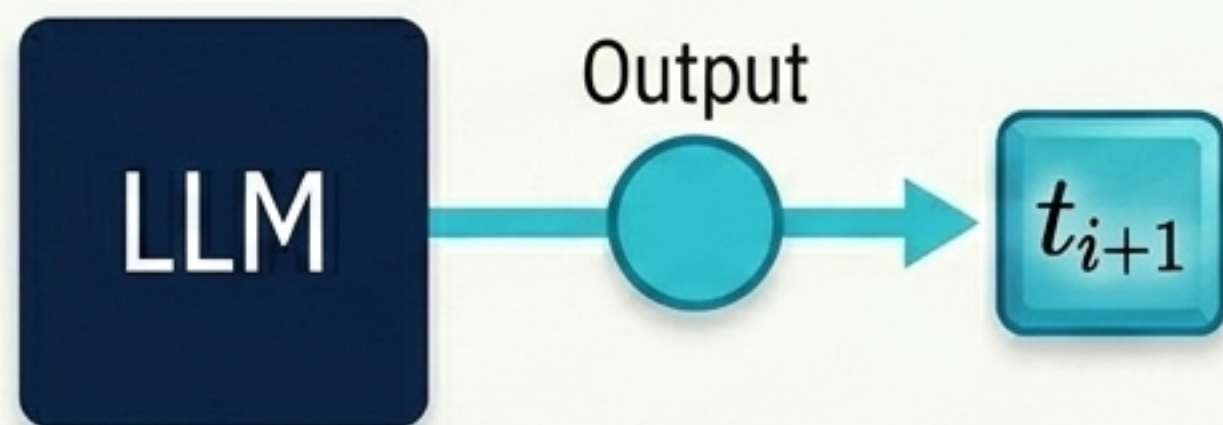
LLM ประมวลผลโทเค็น 1 ตัว แต่ได้รับข้อมูลเทียบเท่า s โทเค็น ทำให้บริบทข้อมูลได้รวดเร็วขึ้นมหาศาล



การซ้อนทับข้อมูลขาออก (Output Superposition)

หลักเลียงข้อจำกัดของ MTP ด้วยเป้าหมายการทำนายแบบกลุ่ม (Bag-of-words summaries)

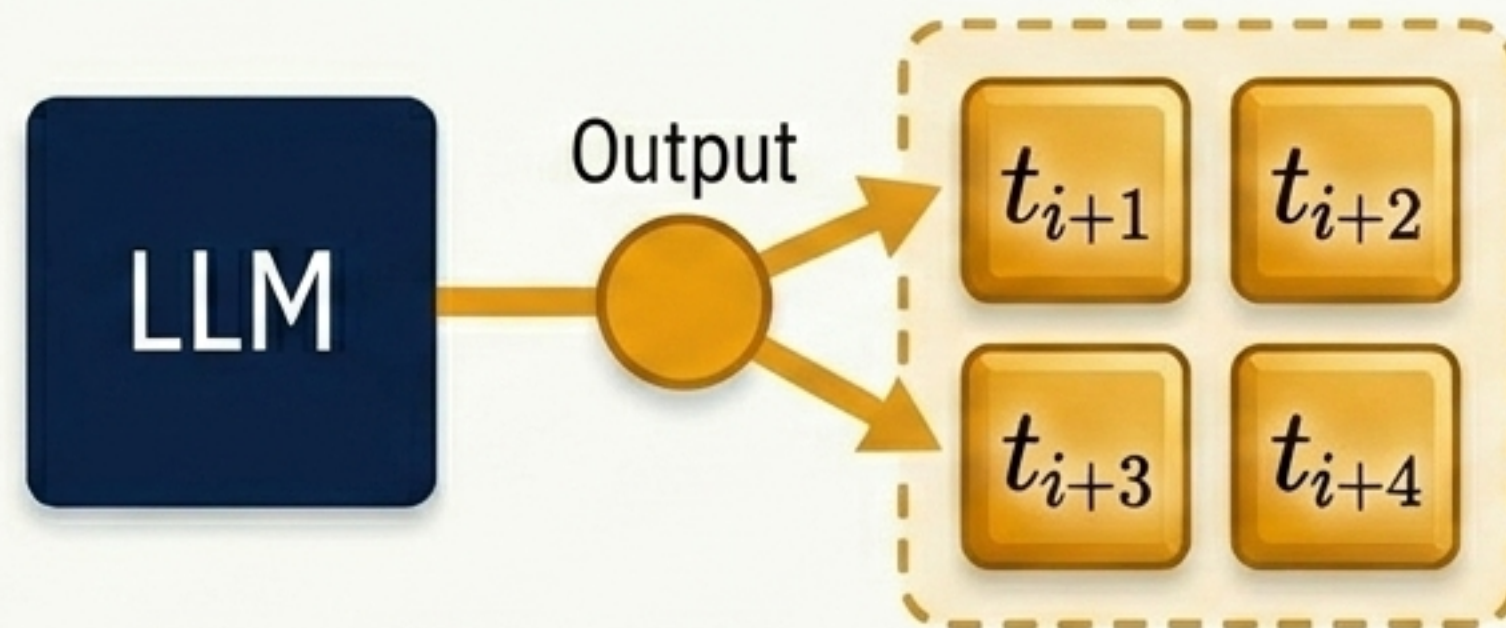
Standard Autoregressive Output (Cyan)



ทำนาย 1 โทเค็นเป้าหมายที่เฉพาะเจาะจงเรียงตามลำดับ
ด้วยเป้าหมาย CE Loss ปกติ

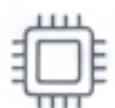
TST Output (Amber)

$$\mathcal{L}_{\text{MCE}} = \frac{1}{|y|} \sum \mathcal{L}_{\text{CE}}(z, y)$$



ทำนายกลุ่มโทเค็นในอนาคตโดยไม่สนใจลำดับ
บังคับกระจายความน่าจะเป็น ($1/s$) เท่าเทียมกันด้วย
Multi-hot Cross-Entropy Loss

หัวใจสำคัญคือการผสานการทำงาน (The Synergy of Full Superposition)



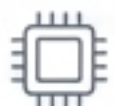
Input-only

เปลี่ยนเฉพาะ Input Granularity
ส่งผลดีต่อความคุ้มค่าของ FLOPs
แต่ไม่เพียงพอ



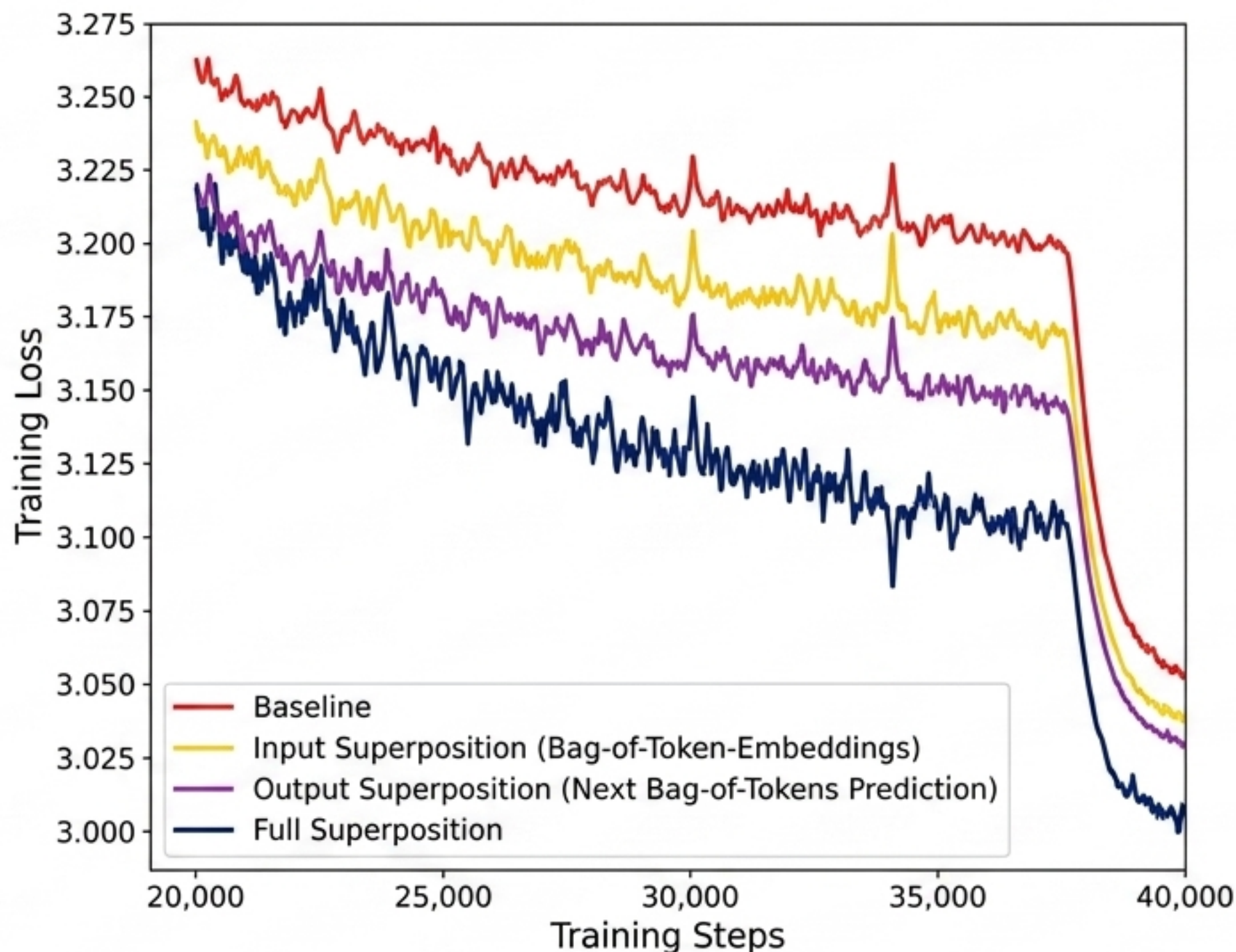
Output-only

เปลี่ยนเฉพาะสัญญาณ Gradient
ช่วยยกระดับการทำนาย แต่ยังไม่ถึงขีดสุด



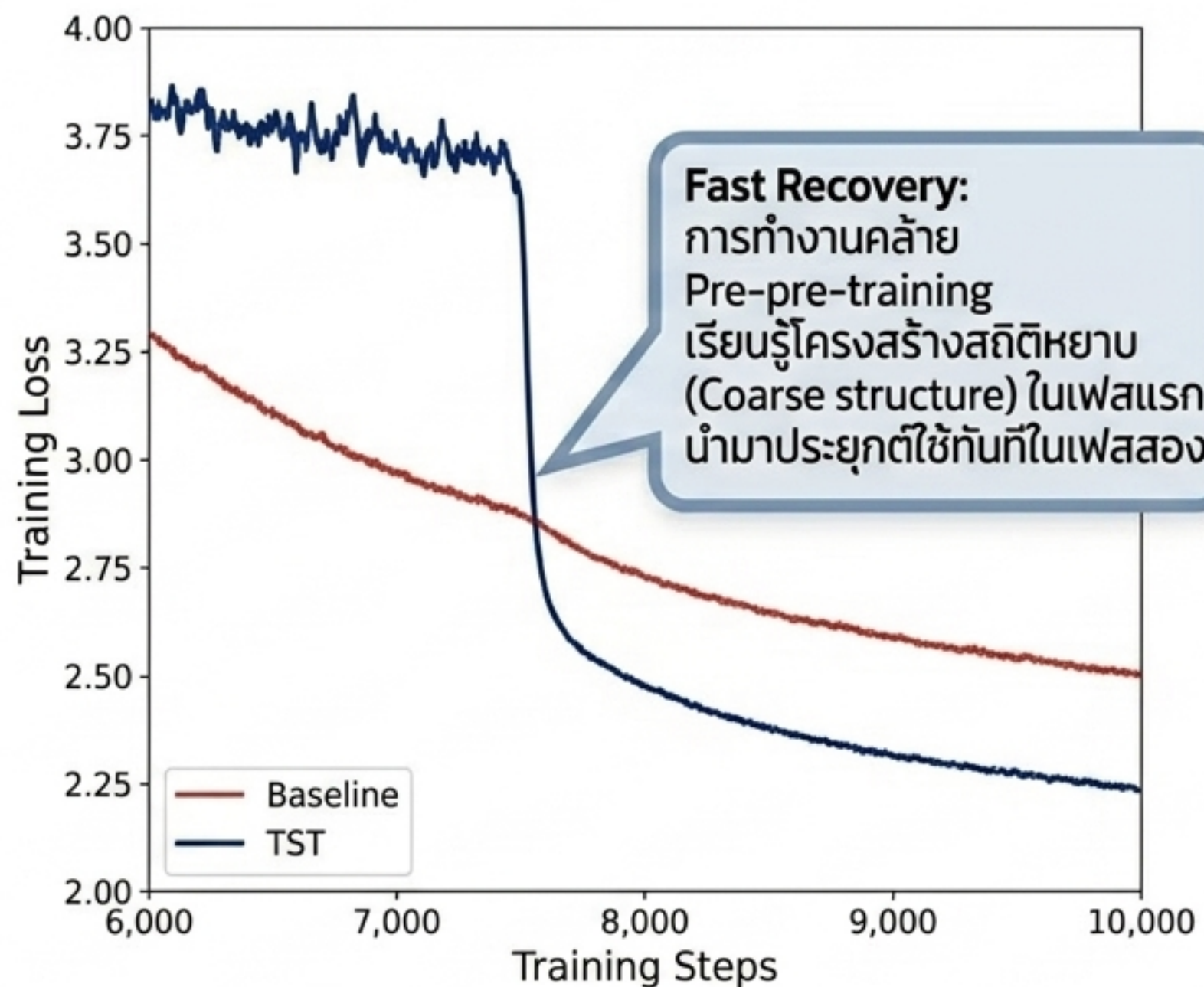
Full Superposition

ข้อสรุป: ผลลัพธ์แบบทวีคูณ
(Multiplicative gain) จะเกิดขึ้น
อย่างสมบูรณ์แบบเมื่อผสานทั้งกลไก
Input และ Output เข้าด้วยกัน

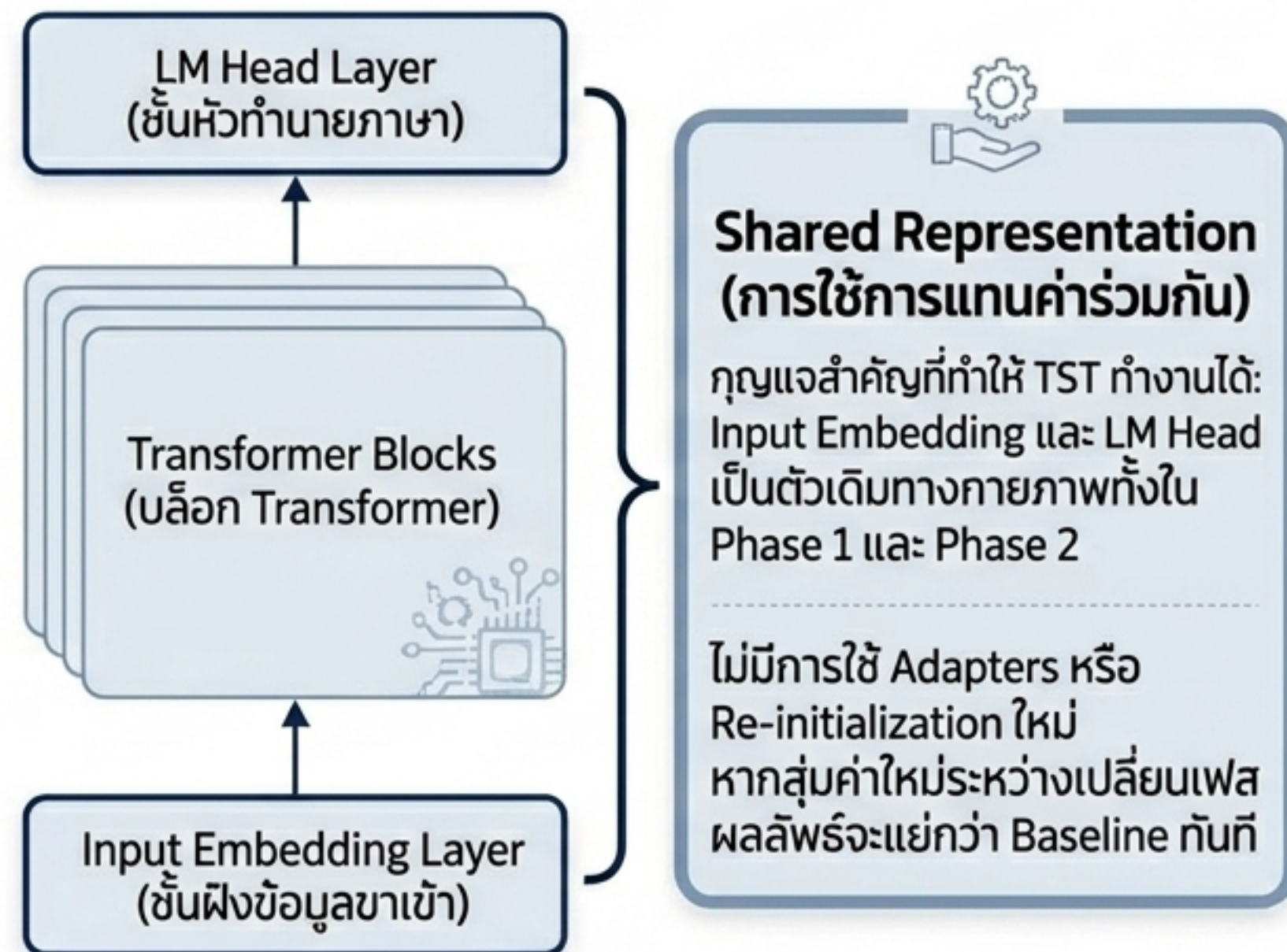


พลวัตของการฝึกสอน: สมมติฐานการจัดเรียงโครงสร้างข้อมูล

The Loss Cliff
(กราฟการลดลงอย่างรวดเร็ว)



Representation Alignment
(การจัดเรียงการแทนค่าข้อมูล)



ความทนทานต่อการขยายสเกลโมเดลระดับเล็กถึงกลาง

สภาพแวดล้อม: ประเมินด้วย TorchTitan แบบ FSDP บน DCLM dataset (Equal-FLOPs)

270M Parameters (100k Steps)

Dense Baseline

Dense Baseline



Dense TST



600M Parameters (20k Steps)

Dense TST

Dense Baseline



Dense TST



3B Parameters (20k Steps)

Dense TST

Dense Baseline



Dense TST



ผลการทดสอบเชิงประจักษ์: TST สามารถเอาชนะ Baseline ได้อย่างสม่ำเสมอในทุกสเกลสถาปัตยกรรม

บทพิสูจน์ขั้นสูงสุด: การลดต้นทุนบนโครงสร้าง 10B MoE

สภาพแวดล้อม: Qwen3-like 10B (1B active) Mixture-of-Experts | สิ้นสุดการเทรนเมื่อค่า Loss เท่ากัน

Standard Baseline

Tokens:

1.05T

Compute:

12,311

B200-Hours

Final Loss:

2.252

ประหยัดทรัพยากรการคำนวณ
(B200-Hours) ไปถึง 61%
โดยที่โมเดลได้เรียนรู้ข้อมูล
มากกว่าเกือบ 2 เท่า

Token-Superposition Training

Tokens:

2.00T

Compute:

4,768

B200-Hours

Final Loss:

2.236

การยกระดับประสิทธิภาพใน Downstream Tasks

ค่า Training Loss ที่ต่ำลง สะท้อนถึงขีดความสามารถที่สูงขึ้นจริงในสถานการณ์ 0-shot evaluation แม้จะเสียบริบทเรื่องลำดับคำไปบ้างในเฟสแรก **แต่ไม่กระทบต่อ** Reasoning

		HellaSwag	ARC-E	ARC-C	MMLU
3B Scale	3B Baseline	57.6	60.6	31.9	31.2
	3B TST	62.4 ↑	66.3 ↑	36.0 ↑	32.8 ↑
10B MoE Scale	10B Baseline	70.1	73.8	46.3	37.4
	10B TST	71.2 ↑	74.2 ↑	47.3 ↑	39.0 ↑

Checklist สำหรับการนำไปใช้งาน (The Drop-In Implementation)

Low Friction, High Impact: สามารถปรับใช้งานบน PyTorch / TorchTitan ได้ทันทีโดยไม่ต้องยุ่งยากกับ Custom CUDA Kernels ที่ซับซ้อน

สิ่งที่ต้องปรับแต่ง (What Changes)

- ✓ **Data Pipeline:** เพิ่ม Script การแบ่ง s-grams และหาค่าเฉลี่ย Embedding
- ✓ **Loss Function:** เปลี่ยนเป็น Multi-hot Cross-Entropy (MCE)
- ✓ **Training Loop:** สร้างเงื่อนไขการตัดจบเฟสช้อนทันที r steps

สิ่งที่ใช้ของเดิม 100% (What Stays the Same)

- ✗ **Model Architecture:** คงเดิมทุกประการ
- ✗ **Tokenizer:** ใช้ชุดคำเดิม
- ✗ **Optimizer & Parallelism:** ใช้ FSDP / AdamW เดิม
- ✗ **Inference Pipeline:** ตั้ง Checkpoint ไปใช้เสมือนเทรนปกติ

บทสรุป: กระบวนทัศน์ใหม่ของการแลกเปลี่ยน Data กับ Compute

สมมติฐานใหม่

- LLM Pre-training ปัจจุบันถูกจำกัดด้วยปริมาณ Compute ไม่ใช่ปริมาณ Data

ทางออกที่ยั่งยืน

- ยอมรับบริโภคข้อมูลมากขึ้นเพื่อแลกกับความเร็วที่เพิ่มขึ้นอย่างก้าวกระโดด ภายใต้ Compute budget ที่จำกัด



ความคุ้มค่าสูงสุด

- เพิ่ม **Training throughput** โดยคงความสมบูรณ์ของ Inference architecture ไว้ได้ดี
- นำมาซึ่งความเร็ว **2.5 เท่า** ที่จับต้องได้จริงในระดับ 10B